


# รายงานผลการปฏิบัติงานสหกิจศึกษา

การพัฒนาเว็บแอปพลิเคชันสำหรับบริหารจัดการและดำเนินการทดสอบซอฟต์แวร์อัตโนมัติ

Development of a Web Application for Automated Software Testing Management and Execution

จัดทำโดย

นายวีรภัทร อุปรัตน์ รหัสนักศึกษา 65122160112

สาขาวิชาวิศวกรรมคอมพิวเตอร์ คณะเทคโนโลยีอุตสาหกรรม

มหาวิทยาลัยราชภัฏลำปาง

ภาคเรียนที่ 2 ปีการศึกษา 2568

| | | |
|---|---|---|
| ชื่อระบบ | : | การพัฒนาเว็บแอปพลิเคชันสำหรับบริหารจัดการและดำเนินการทดสอบซอฟต์แวร์อัตโนมัติ<br>Development of a Web Application for Automated Software Testing Management and Execution |
| ชื่อผู้พัฒนา | : | นายวีรภัทร อุปรัตน์ |
| คณะ | : | เทคโนโลยีอุตสาหกรรม |
| มหาวิทยาลัย | : | มหาวิทยาลัยราชภัฏลำปาง |
| ปีการศึกษา | : | 2568 |

## บทคัดย่อ


การทดสอบซอฟต์แวร์แบบดั้งเดิม (Manual Testing) เป็นกระบวนการที่ใช้เวลาดำเนินการสูง เกิดความผิดพลาดจากมนุษย์ (Human Error) ได้ง่าย และมีความท้าทายในการจัดเก็บหลักฐานการทดสอบ (Evidence) เช่น ภาพถ่ายหน้าจอและบันทึกการทำงาน (Logs) ซึ่งมักกระจัดกระจายและยากต่อการตรวจสอบย้อนหลัง โครงงานนี้จึงมีวัตถุประสงค์เพื่อพัฒนา "ระบบเว็บแอปพลิเคชันสำหรับบริหารจัดการและดำเนินการทดสอบซอฟต์แวร์อัตโนมัติ" (Automated Software Testing Management Web Application) เพื่อเพิ่มประสิทธิภาพและลดระยะเวลาในกระบวนการทดสอบขององค์กร

ระบบถูกออกแบบและพัฒนาด้วยสถาปัตยกรรมแบบแยกส่วน (Microservices Architecture) ประกอบด้วย 3 ส่วนหลัก ได้แก่ 1) ส่วนหน้าบ้าน (Frontend) พัฒนาด้วย React ทำหน้าที่เป็นส่วนติดต่อผู้ใช้งาน (User Interface) ให้สามารถสั่งรันการทดสอบและดูรายงานผล 2) ส่วนหลังบ้าน (Backend) พัฒนาด้วยภาษา Go (Fiber Framework) ทำหน้าที่เป็นศูนย์กลาง (Orchestrator) ในการประมวลผลคำสั่ง จัดการข้อมูลความลับ (Environment Variables) และจัดการระบบไฟล์หลักฐาน 3) ส่วนปฏิบัติการทดสอบ (Automation Engine) พัฒนาด้วย Node.js และ Playwright ทำหน้าที่จำลองพฤติกรรมการใช้งานบนเบราว์เซอร์จริง

ผลการดำเนินงานพบว่า ระบบสามารถอ่านโครงสร้างการทดสอบจากไฟล์การตั้งค่า (JSON Configuration) มาสร้างเป็นหน้าจอโต้ตอบแบบไดนามิก ผู้ใช้งานสามารถสั่งรันชุดการทดสอบผ่าน

หน้าเว็บได้ทันทีด้วยการคลิกเพียงครั้งเดียว (Zero-code Execution) เมื่อสิ้นสุดการทดสอบ ระบบจะทำการรวบรวมหลักฐาน ทั้งสถานะการรัน (Pass/Fail) บันทึกการทำงาน และภาพถ่ายหน้าจอแบบอัตโนมัติ โดยจัดเก็บแยกตามรอบการทำงาน (Run ID) ไว้ในโฟลเดอร์ Artifacts อย่างเป็นระบบ และสามารถเรียกดูประวัติย้อนหลังผ่านหน้าเว็บได้ทันที นอกจากนี้ ระบบยังถูกออกแบบด้วยแนวคิด Data-Driven Framework ทำให้มีความยืดหยุ่นสูง (Scalability) พร้อมรองรับการเพิ่มชุดการทดสอบของระบบซอฟต์แวร์อื่นๆ ในอนาคตได้โดยไม่ต้องแก้ไขโครงสร้างโปรแกรมหลัก ซึ่งช่วยยกระดับมาตรฐานการทดสอบซอฟต์แวร์ขององค์กรให้รวดเร็วและเป็นระบบมากยิ่งขึ้น

# กิตติกรรมประกาศ

โครงงานเรื่อง การพัฒนาเว็บแอปพลิเคชันเพื่อบริหารจัดการและดำเนินการทดสอบซอฟต์แวร์อัตโนมัติ ฉบับนี้ สำเร็จลุล่วงไปได้ด้วยความกรุณาของ ของ นางอรุณนี พ่วงบทโพธิ์ ตำแหน่ง หัวหน้างานพัฒนาซอฟต์แวร์ (Software Manager) มูลนิธิแม่ฟ้าหลวงในพระบรมราชูปถัมภ์ ที่ได้สละเวลาให้คำปรึกษา ชี้แนะแนวทางการทำงาน ตลอดจนถ่ายทอดความรู้เรื่องขั้นตอนและวิธีการใช้งานเครื่องมือต่างๆ ในการพัฒนาระบบอย่างใกล้ชิด จนส่งผลให้โครงงานนี้สำเร็จสมบูรณ์ด้วยดี ผู้จัดทำขอกราบขอบพระคุณเป็นอย่างสูงมา ณ โอกาสนี้

คณะผู้จัดทำ

16 กุมภาพันธ์ 2569

# สารบัญ

หน้า

กิตติกรรมประกาศ

บทคัดย่อ

สารบัญ

สารบัญตาราง

สารบัญรูปภาพ

บทที่ 1 บทนำ

1.1 ความเป็นมาและความสำคัญของโครงงาน

1.2 วัตถุประสงค์ของโครงงาน

1.3 ขอบเขตของโครงงาน

1.4 แผนการดำเนินงาน

บทที่ 2 ข้อมูลพื้นฐานของหน่วยงาน และแนวคิดทฤษฎีที่นำมาใช้ในการปฏิบัติงาน

2.1 ข้อมูลพื้นฐานของหน่วยงาน

2.1.1 ชื่อหน่วยงาน / ที่ตั้งสำนักงาน

2.1.2 ประวัติความเป็นมา / ประวัติการก่อตั้ง

2.1.3 การดำเนินธุรกิจ / ลักษณะธุรกิจ / ผลิตภัณฑ์หรือบริการ

2.1.4 โครงสร้างหน่วยงาน

2.1.5 บทบาทหน้าที่ความรับผิดชอบของบุคลากรแต่ละฝ่าย

2.1.6 ตำแหน่งงานและลักษณะงานที่นักศึกษาได้รับมอบหมาย

2.1.7 ระยะเวลาที่ปฏิบัติงานสหกิจศึกษา

2.1.8 การจัดสวัสดิการ เบี้ยเลี้ยง ทั้งรูปแบบตัวเงินหรือสวัสดิการ (in cash หรือ in kind)

2.2 แนวคิด / ทฤษฎีที่เกี่ยวข้องและนำมาใช้ในการปฏิบัติงาน

2.2.1 แนวคิดการทดสอบซอฟต์แวร์อัตโนมัติ (Automated Software Testing)

2.2.2 ภาษา Go และ Fiber Framework

2.2.3 ไลบรารี React และการพัฒนาส่วนติดต่อผู้ใช้งาน

2.2.4 เครื่องมือ Playwright และ Node.js

2.2.5 โครงสร้างข้อมูล JSON และสถาปัตยกรรม REST API

บทที่ 3 กระบวนการปฏิบัติงานสหกิจศึกษา

3.1 ขั้นตอนการปฏิบัติงาน (โครงงาน/งานวิจัย)

3.1.1 การศึกษาความต้องการและออกแบบสถาปัตยกรรมระบบ (System Architecture Design)

3.1.2 การพัฒนาระบบส่วนหน้าบ้านและหลังบ้าน (Frontend & Backend Development)

3.1.3 การพัฒนาสคริปต์ทดสอบอัตโนมัติ (Automation Scripting)

3.1.4 การทดสอบและปรับปรุงการทำงานของระบบ (Testing and Deployment)

3.2 การดำเนินงานมีความถูกต้อง มีระเบียบแบบแผน และทำให้นักศึกษามีโอกาสใช้วิชาความรู้/ทักษะตามที่เรียนมา

3.3 ความพึงพอใจของสถานประกอบการต่อโครงงาน

3.4 เป็นงานที่นำไปใช้ประโยชน์ได้อย่างเป็นรูปธรรมในสถานประกอบการ

บทที่ 4 ผลการปฏิบัติงาน

4.1 ผลการพัฒนาระบบส่วนติดต่อผู้ใช้งาน (Frontend - React)

4.2 ผลการพัฒนาระบบจัดการหลังบ้าน (Backend - Go & Fiber)

4.3 ผลการพัฒนาระบบปฏิบัติการทดสอบอัตโนมัติ (Automation Engine - Playwright)

4.4 ผลลัพธ์การจัดเก็บหลักฐานการทดสอบ (Testing Artifacts & Logs)

บทที่ 5 สรุปผลการปฏิบัติงานสหกิจศึกษา

5.1 สรุปผลโครงงานสหกิจศึกษา

5.2 สิ่งที่ได้รับจากการปฏิบัติงานสหกิจศึกษา

5.3 ปัญหาและอุปสรรคที่พบจากการปฏิบัติงาน

5.4 วิธีแก้ไขปัญหาที่พบจากการปฏิบัติงาน (ถ้ามี)

5.5 ความรู้และทักษะใหม่ที่ได้รับระหว่างปฏิบัติงานสหกิจศึกษา

5.6 ข้อเสนอแนะสำหรับการปฏิบัติงานสหกิจศึกษาสำหรับนักศึกษารุ่นต่อไป

บรรณานุกรม

ภาคผนวก

ภาคผนวก ก. คู่มือการใช้งานระบบ (User Manual)

ภาคผนวก ข. ตัวอย่างโค้ดที่สำคัญ (Source Code Snippets)

# สารบัญตาราง

หน้า

ตารางที่

แผนการดำเนินงาน

# สารบัญรูปภาพ

# บทที่ 1
# บทนำ

## 1.1 ความเป็นมาและความสำคัญของโครงงาน

ในปัจจุบัน เทคโนโลยีสารสนเทศและระบบซอฟต์แวร์ได้เข้ามามีบทบาทสำคัญในการขับเคลื่อนการดำเนินงานขององค์กรทุกภาคส่วน รวมถึงองค์กรไม่แสวงหากำไรอย่าง มูลนิธิแม่ฟ้าหลวงในพระบรมราชูปถัมภ์ เพื่อให้การบริหารจัดการข้อมูลภายใน การให้บริการ และการดำเนินกิจกรรมต่างๆ ของมูลนิธิเป็นไปอย่างรวดเร็ว โปร่งใส และมีประสิทธิภาพสูงสุด การพัฒนาระบบซอฟต์แวร์ภายในองค์กร (Internal Systems) เช่น ระบบเบิกจ่าย (Reimbursement System) หรือระบบจัดการข้อมูลส่วนอื่นๆ จึงต้องมีความถูกต้องและเสถียรภาพอยู่เสมอ

อย่างไรก็ตาม ก่อนที่ระบบซอฟต์แวร์แต่ละส่วนจะถูกนำไปใช้งานจริง หรือเมื่อมีการอัปเดตระบบเพิ่มเติม จำเป็นต้องผ่านกระบวนการทดสอบซอฟต์แวร์ (Software Testing) เพื่อค้นหาข้อผิดพลาด ปัญหาที่พบในการปฏิบัติงานปัจจุบันคือ กระบวนการทดสอบส่วนใหญ่ยังคงพึ่งพาการทดสอบด้วยมือ (Manual Testing) ซึ่งบุคลากรหรือผู้ทดสอบ (QA) ต้องจำลองการทำงานด้วยตนเองซ้ำๆ (Repetitive Tasks) ในทุกหน้าจอ การดำเนินการในลักษณะนี้ไม่เพียงแต่ใช้เวลาและทรัพยากรบุคคลสูง แต่ยังเสี่ยงต่อการเกิดความผิดพลาดจากตัวบุคคล (Human Error) นอกจากนี้ ความท้าทายที่สำคัญอีกประการคือ “การจัดเก็บหลักฐานการทดสอบ (Testing Evidence)” เช่น ภาพถ่ายหน้าจอ (Screenshots) และบันทึกสถานะการทำงาน (Logs) ซึ่งมักกระจัดกระจาย จัดการได้ยาก และใช้เวลามากในการรวบรวมเพื่อทำรายงานสรุปผล สำหรับองค์กรอย่างมูลนิธิที่มีบุคลากรจำกัด การสูญเสียเวลาไปกับงานทดสอบซอฟต์แวร์แบบดั้งเดิม ย่อมส่งผลกระทบต่อเวลาที่ควรนำไปใช้ในการพัฒนางานหลักด้านอื่นๆ

จากปัญหาและความท้าทายดังกล่าว ผู้จัดทำจึงมีแนวคิดที่จะนำเทคโนโลยีการทดสอบอัตโนมัติ (Automated Testing) มาประยุกต์ใช้เพื่อแก้ปัญหาอย่างเป็นรูปธรรม โดยได้ริเริ่มโครงงาน “การพัฒนาเว็บแอปพลิเคชันเพื่อบริหารจัดการและดำเนินการทดสอบซอฟต์แวร์อัตโนมัติ” โครงงานนี้เป็นการสร้างแพลตฟอร์มศูนย์กลาง (Centralized Testing Hub) ที่เชื่อมต่อการทำงานระหว่างส่วนติดต่อผู้ใช้ (Frontend) ที่พัฒนาด้วย React, ระบบหลังบ้าน (Backend) ที่ควบคุมด้วยภาษา Go และเครื่องมือจำลองพฤติกรรมเว็บเบราว์เซอร์อย่าง Playwright เข้าไว้ด้วยกันความสำคัญและจุดเด่นของโครงงานนี้คือ การเปลี่ยนกระบวนการเขียนโค้ดทดสอบที่ซับซ้อน ให้กลายเป็นระบบที่ผู้ใช้งาน

สามารถสั่งดำเนินการทดสอบได้ผ่านหน้าเว็บ (Zero-code Execution) เพียงแค่คลิกปุ่ม ระบบจะทำการจำลองพฤติกรรมการคลิก กรอกข้อมูล และตรวจสอบความถูกต้องของระบบปลายทางโดยอัตโนมัติ พร้อมทั้งความสามารถในการจับภาพหน้าจอและบันทึกข้อผิดพลาด (Artifacts & Logs) แยกตามรอบการทดสอบอย่างเป็นระเบียบ นอกจากนี้ โครงสร้างของระบบยังถูกออกแบบด้วยแนวคิด Data-Driven Framework ทำให้มีความยืดหยุ่นสูง รองรับการเพิ่มชุดการทดสอบสำหรับระบบอื่นๆ ของมูลนิธิในอนาคตได้ทันที โครงงานนี้จึงไม่เพียงแต่ช่วยลดระยะเวลาและข้อผิดพลาดในการทดสอบซอฟต์แวร์ แต่ยังช่วยยกระดับมาตรฐานการพัฒนาเทคโนโลยีภายในมูลนิธิให้มีความรวดเร็ว แม่นยำ และตรวจสอบย้อนหลังได้ในระยะยาว

ดังนั้น โครงงานนี้จึงมีความสำคัญอย่างยิ่ง โดยคาดหวังว่าระบบที่พัฒนาขึ้นจะมอบประโยชน์ให้แก่มูลนิธิอย่างเป็นรูปธรรม ทั้งในการช่วยลดระยะเวลาและลดข้อผิดพลาดที่เกิดจากการทดสอบด้วยแรงงานคน รวมถึงมีระบบจัดเก็บหลักฐานอัตโนมัติที่ช่วยให้บุคลากรสามารถตรวจสอบประวัติย้อนหลังได้อย่างรวดเร็วและเป็นระบบ ตลอดจนมีโครงสร้างระบบที่มีความยืดหยุ่นสูง ซึ่งจะช่วยลดภาระงานซ้ำซ้อน และยกระดับกระบวนการพัฒนาซอฟต์แวร์ภายในองค์กรให้มีประสิทธิภาพ พร้อมรองรับการขยายตัวในอนาคตได้อย่างยั่งยืน

**1.2 วัตถุประสงค์ของโครงงาน**

การจัดทำโครงงาน “การพัฒนาเว็บแอปพลิเคชันเพื่อบริหารจัดการและดำเนินการทดสอบซอฟต์แวร์อัตโนมัติ” มีวัตถุประสงค์หลักดังต่อไปนี้เพื่อพัฒนาเว็บแอปพลิเคชันสำหรับบริหารจัดการการทดสอบซอฟต์แวร์ ที่มีส่วนติดต่อผู้ใช้งาน (User Interface) ให้สามารถสั่งรันการทดสอบและติดตามสถานะได้อย่างสะดวกผ่านหน้าเว็บเพื่อลดระยะเวลาและข้อผิดพลาด (Human Error) ที่เกิดจากระบวนการทดสอบซอฟต์แวร์ด้วยแรงงานคน (Manual Testing) แบบดั้งเดิมเพื่อพัฒนาระบบจัดเก็บและรวบรวมหลักฐานการทดสอบอัตโนมัติ (Automated Artifacts Collection) เช่น ภาพถ่ายหน้าจอ (Screenshots) และบันทึกการทำงาน (Logs) ให้เป็นหมวดหมู่ตามรอบการทดสอบเพื่อความสะดวกในการตรวจสอบย้อนหลังเพื่อออกแบบสถาปัตยกรรมระบบทดสอบให้มีความยืดหยุ่น (Scalability) ด้วยแนวคิดขับเคลื่อนด้วยข้อมูล (Data-Driven Framework) ซึ่งช่วยให้มูลนิธิสามารถเพิ่มชุดการทดสอบสำหรับระบบงานอื่นๆ ในอนาคตได้ โดยไม่ต้องแก้ไขโครงสร้างซอร์สโค้ด (Source Code) หลัก

**1.3 ขอบเขตของโครงงาน**

การจัดทำโครงงาน “การพัฒนาเว็บแอปพลิเคชันเพื่อบริหารจัดการและดำเนินการทดสอบซอฟต์แวร์อัตโนมัติ” ได้กำหนดขอบเขตของการศึกษาและพัฒนาระบบออกเป็น 3 ด้านหลัก ดังนี้

1.3.1 ขอบเขตด้านผู้ใช้งาน (User Scope) ผู้ใช้งานระบบ (เช่น นักพัฒนาซอฟต์แวร์, ผู้ทดสอบ
ระบบ (QA) หรือบุคลากรของมูลนิธิ) สามารถเข้าถึงหน้าเว็บแอปพลิเคชันเพื่อดูรายการชุดการทดสอบ (Test Suites) สั่งดำเนินการทดสอบ (Run Test) แบบเฉพาะกิจ (Ad-hoc) ดูสถานะการทำงาน และเรียกดูประวัติการทดสอบย้อนหลังพร้อมหลักฐานภาพถ่ายหน้าจอได้

1.3.2 ขอบเขตด้านระบบงาน (System Scope) ระบบถูกแบ่งการทำงานออกเป็น 3 ส่วนหลัก ได้แก่

ส่วนติดต่อผู้ใช้งาน (Frontend) พัฒนาด้วย React ร่วมกับไลบรารี Mantine React Table ทำหน้าที่แสดงผลตารางรายการทดสอบที่ดึงข้อมูลจากไฟล์การตั้งค่า (JSON) รองรับการโต้ตอบกับ

ผู้ใช้ด้วยปุ่มสั่งการ รวมถึงมีหน้าต่างแสดงประวัติการทดสอบ (History View) และหลักฐานการทำงาน

ส่วนหลังบ้านและศูนย์กลางควบคุม (Backend & Orchestrator) พัฒนาด้วยภาษา Go ทำหน้าที่เป็น API Server รับคำสั่งจากหน้าเว็บ จัดการการแทรกข้อมูลตัวแปรความลับ (Environment Variables) เช่น ชื่อผู้ใช้และรหัสผ่านเข้าสู่ชุดทดสอบ ควบคุมการสร้างโฟลเดอร์จัดเก็บหลักฐาน (Artifacts) อัตโนมัติในแต่ละรอบการรัน (Run-ID) และมีระบบจัดการโปรเซส (Process Management) เพื่อควบคุมการเริ่มและยกเลิกการทดสอบอย่างปลอดภัย

ส่วนปฏิบัติการทดสอบอัตโนมัติ (Automation Engine) พัฒนาด้วย Node.js และ Playwright ทำหน้าที่อ่านคำสั่งจากไฟล์สคริปต์ (JSON Scenario) เพื่อจำลองพฤติกรรมการใช้งานบนเว็บเบราว์เซอร์จริง เช่น การกรอกข้อมูล การคลิก และการจับภาพหน้าจอ (Screenshots) ตามขั้นตอนที่กำหนด หรือเมื่อเกิดข้อผิดพลาด (Error Handling) พร้อมทั้งสรุปผลการรันเป็นไฟล์รายงาน (result.json)

ข้อจำกัดของโครงงาน (Limitations)

ระบบปฏิบัติการทดสอบ (Automation Engine) ในโครงงานนี้ ถูกออกแบบมาเพื่อรองรับการทดสอบระบบประเภท เว็บแอปพลิเคชัน (Web Application) เป็นหลัก (ซึ่งรวมถึงการจำลองการทดสอบเว็บบนเบราว์เซอร์ของอุปกรณ์เคลื่อนที่ หรือ Responsive Web) แต่ขอบเขตยังไม่ครอบคลุมถึงการทดสอบแอปพลิเคชันที่ต้องติดตั้งบนระบบปฏิบัติการสมาร์ทโฟนโดยตรง (Native Mobile Application บน iOS และ Android) การเพิ่มชุดการทดสอบใหม่ (New Test Cases) ในปัจจุบันใช้วิธีการเขียนโครงสร้างคำสั่งผ่านไฟล์ JSON (Configuration-driven) โดยตรง ยังไม่มีส่วนติดต่อผู้ใช้งานแบบกราฟิก (GUI) สำหรับการลากวาง (Drag and Drop) เพื่อสร้างชุดการทดสอบบนหน้าเว็บ

**1.4 แผนการดำเนินการ**

**ตารางที่ 1.1** ตารางขั้นตอนและระยะเวลาในการดำเนินงาน

| ขั้นตอนการดำเนินงาน | เดือน - ปี | | | |
|---|---|---|---|---|
| | พ.ศ. 2568 | | พ.ศ. 2569 | |
| | พ.ย | ธ.ค | ม.ค | ก.พ |
| ศึกษาความต้องการและออกแบบสถาปัตยกรรมระบบ | ✔ | | | |
| พัฒนาระบบส่วนหน้าบ้าน (Frontend - React) | | ✔ | | |
| พัฒนาระบบส่วนหลังบ้าน (Backend - Go & Fiber) | | ✔ | ✔ | |
| พัฒนาสคริปต์ทดสอบอัตโนมัติ (Playwright) | | | ✔ | ✔ |
| ทดสอบการทำงานระบบรวม และแก้ไขข้อผิดพลาด | | | | ✔ |
| จัดทำคู่มือและรายงานสรุปผลสหกิจศึกษา | | | | ✔ |

# บทที่ 2
# ข้อมูลพื้นฐานของหน่วยงาน และแนวคิดทฤษฎีที่นำมาใช้ในการปฏิบัติงาน

## 2.1 ข้อมูลพื้นฐานของหน่วยงาน

### 2.1.1 ชื่อหน่วยงาน / ที่ตั้งสำนักงานหรือหน่วยงาน

มูลนิธิแม่ฟ้าหลวง ในพระบรมราชูปถัมภ์ (โครงการพัฒนาดอยตุง จ.เชียงราย) อาคารอเนกประสงค์ พระตำหนักดอยตุง อำเภอแม่ฟ้าหลวง จังหวัดเชียงราย 57240

### 2.1.2 ประวัติความเป็นมา / ประวัติการก่อตั้ง

มูลนิธิแม่ฟ้าหลวง ในพระบรมราชูปถัมภ์ ก่อตั้งขึ้นจากพระราชปณิธานของ สมเด็จพระศรีนครินทราบรมราชชนนี (สมเด็จย่า) ที่ทรงตระหนักถึงปัญหาความยากจน การปลูกฝิ่น และการทำลายทรัพยากรป่าไม้บนพื้นที่สูง จึงทรงริเริ่ม "โครงการพัฒนาดอยตุง (พื้นที่ทรงงาน) อันเนื่องมาจากพระราชดำริ" ขึ้นในปี พ.ศ. 2530 ด้วยพระราชหฤทัยที่มุ่งมั่นตามแนวคิด "ฉันจะปลูกป่าบนดอยตุง" เพื่อยกระดับคุณภาพชีวิตของชาวไทยภูเขา ทั้งในด้านเศรษฐกิจ สังคม และสิ่งแวดล้อม ควบคู่ไปกับการฟื้นฟูผืนป่า

ต่อมาเมื่อสมเด็จย่าเสด็จสวรรคตในปี พ.ศ. 2538 พระบาทสมเด็จพระบรมชนกาธิเบศร มหาภูมิพลอดุลยเดชมหาราช บรมนาถบพิตร ทรงพระกรุณาโปรดเกล้าฯ รับมูลนิธิไว้ในพระบรมราชูปถัมภ์ และปัจจุบัน สมเด็จพระกนิษฐาธิราชเจ้า กรมสมเด็จพระเทพรัตนราชสุดาฯ สยามบรมราชกุมารี ทรงเป็นนายกกิตติมศักดิ์ โดยมูลนิธิฯ ยังคงสานต่อแนวทางการพัฒนาทางเลือกในการดำรงชีวิตที่ยั่งยืน (Sustainable Alternative Livelihood Development) อย่างต่อเนื่อง

### 2.1.3 การดำเนินธุรกิจ / ลักษณะธุรกิจ / ผลิตภัณฑ์หรือบริการ

มูลนิธิแม่ฟ้าหลวงฯ ดำเนินงานในรูปแบบ ธุรกิจเพื่อสังคม (Social Enterprise) เพื่อนำรายได้กลับไปสนับสนุนการพัฒนาชุมชนอย่างยั่งยืน โดยมีผลิตภัณฑ์และบริการหลักภายใต้แบรนด์ "ดอยตุง" (DoiTung) ซึ่งแบ่งออกเป็น 5 กลุ่มธุรกิจหลัก ได้แก่

งานหัตถกรรม (Handicrafts) ผลิตภัณฑ์เสื้อผ้าทอมือ ผ้าพันคอ และงานเซรามิกที่เป็นมิตรกับสิ่งแวดล้อม

อาหารแปรรูป (Macadamia & Processed Food) ผลิตภัณฑ์จากแมคคาเดเมีย และผลผลิตทางการเกษตรของชุมชน

ธุรกิจกาแฟและคาเฟ่ (Café DoiTung) กาแฟอาราบิก้าคุณภาพสูงที่ปลูกใต้ร่มเงาไม้ใหญ่ พร้อมให้บริการผ่านสาขาร้านกาแฟดอยตุง

เกษตรกรรม (Agriculture) ไม้ดอก ไม้ประดับ และการเพาะเลี้ยงพันธุ์ไม้การท่องเที่ยว (Tourism) แหล่งท่องเที่ยวเชิงนิเวศและวัฒนธรรม เช่น พระตำหนักดอยตุง สวนแม่ฟ้าหลวง และหอแห่งแรงบันดาลใจ

**2.1.4 โครงสร้างหน่วยงาน**

มูลนิธิแม่ฟ้าหลวงฯ มีโครงสร้างการบริหารงานที่ชัดเจน โดยมีคณะกรรมการมูลนิธิเป็นผู้กำหนดนโยบาย (Policy Level) และมีคณะผู้บริหารระดับสูง นำโดย ประธานเจ้าหน้าที่บริหาร (CEO) ทำหน้าที่ขับเคลื่อนแผนงาน ทั้งนี้ โครงสร้างภายในถูกแบ่งออกเป็นสายงานหลักตามพันธกิจ เช่น สายงานพัฒนาธุรกิจและแบรนด์ดอยตุง, สายงานพัฒนาชุมชนและสิ่งแวดล้อม (Nature-based Solutions), ฝ่ายสนับสนุนองค์กร เช่น ฝ่ายเทคโนโลยีสารสนเทศ (IT), ฝ่ายบัญชีและการเงิน และฝ่ายทรัพยากรบุคคล เพื่อให้การทำงานในองค์กรเป็นไปอย่างบูรณาการ

**2.1.5 บทบาทหน้าที่ความรับผิดชอบของบุคลากรแต่ละฝ่าย**

สายงานพัฒนาธุรกิจ รับผิดชอบการผลิต การตลาด และการจัดจำหน่ายสินค้าแบรนด์ดอยตุง สายงานพัฒนาชุมชนและสิ่งแวดล้อม ดูแลโครงการปลูกป่า การจัดการน้ำ และยกระดับคุณภาพชีวิตชาวบ้านฝ่ายเทคโนโลยีสารสนเทศ (IT Department) (ซึ่งเป็นส่วนที่ผู้จัดทำได้เข้าไปปฏิบัติงาน) มีหน้าที่ในการออกแบบ พัฒนา และดูแลระบบซอฟต์แวร์แอปพลิเคชันภายใน (Internal Systems) เช่น ระบบงานบุคคล ระบบเบิกจ่าย ตลอดจนวางโครงสร้างพื้นฐานทางเครือข่าย เพื่อสนับสนุนให้ทุกฝ่ายงานของมูลนิธิสามารถดำเนินงานได้อย่างมีประสิทธิภาพและปลอดภัย

**2.1.6 ตำแหน่งงานและลักษณะงานที่นักศึกษาได้รับมอบหมาย**

ตำแหน่งงาน : Fullstack Developer

ลักษณะงานที่ได้รับมอบหมาย :

1. ศึกษาความต้องการและออกแบบสถาปัตยกรรมระบบทดสอบอัตโนมัติ
2. พัฒนาเว็บแอปพลิเคชันส่วนหน้าบ้าน (Frontend) ด้วย React
3. พัฒนาระบบ API หลังบ้าน (Backend) ด้วยภาษา Go (Fiber Framework)
4. เขียนสคริปต์จำลองการทดสอบระบบด้วย Playwright และจัดการระบบจัดเก็บหลักฐานการทดสอบ

**2.1.7 ระยะเวลาที่ปฏิบัติงานสหกิจศึกษา**

ผู้จัดทำได้เข้าปฏิบัติงานสหกิจตั้งแต่วันที่ 3 พฤศจิกายน พ.ศ.2568 ถึงวันที่ 20 กุมภาพันธ์ พ.ศ.2569 รวมระยะเวลาทั้งสิ้น 16 สัปดาห์

**2.1.8 การจัดสวัสดิการ เบี้ยเลี้ยง ทั้งรูปแบบตัวเงินหรือสวัสดิการ**

ค่าส่วนลดการซื้อของในโครงการดอยตุง อาหาร ลด 20% , เครื่องดื่ม 50% , สินค้าต่างๆ 50%

## 2.2 แนวคิด / ทฤษฎีที่เกี่ยวข้องและนำมาใช้ในการปฏิบัติงาน

ในการจัดทำโครงงาน "การพัฒนาเว็บแอปพลิเคชันเพื่อบริหารจัดการและดำเนินการทดสอบซอฟต์แวร์อัตโนมัติ" ผู้จัดทำได้ศึกษา ค้นคว้า และนำแนวคิด ทฤษฎี ตลอดจนเทคโนโลยีต่างๆ มาประยุกต์ใช้ในการพัฒนาระบบ ดังมีรายละเอียดต่อไปนี้

**2.2.1 แนวคิดการทดสอบซอฟต์แวร์อัตโนมัติ (Automated Software Testing)**

ทฤษฎี : การทดสอบซอฟต์แวร์อัตโนมัติ คือกระบวนการใช้เครื่องมือหรือสคริปต์โปรแกรมคอมพิวเตอร์เพื่อควบคุมการทำงานและตรวจสอบผลลัพธ์ของซอฟต์แวร์เปรียบเทียบกับผลลัพธ์ที่คาดหวัง โดยไม่ต้องใช้แรงงานคน (Manual Testing) ในการทำซ้ำ แนวคิดนี้ช่วยลดระยะเวลา ลดความผิดพลาดจากมนุษย์ (Human Error) และเพิ่มความครอบคลุมในการทดสอบ (Test Coverage)

การนำมาประยุกต์ใช้ : ผู้จัดทำได้นำแนวคิดนี้มาเป็นแกนหลักของโครงงาน โดยเปลี่ยนกระบวนการทดสอบระบบเว็บแอปพลิเคชันของมูลนิธิจากการใช้พนักงาน QA นั่งกดทีละหน้าจอ เป็นการรันผ่านระบบอัตโนมัติ พร้อมทั้งประยุกต์ใช้แนวคิด Data-Driven Testing โดยแยกข้อมูลชุดทดสอบออกมาเป็นไฟล์คอนฟิก ทำให้สามารถแก้ไขหรือเพิ่มชุดทดสอบได้โดยไม่ต้องรื้อเขียนโปรแกรมใหม่

**2.2.2 ภาษา Go (Golang) และ Fiber Framework**

ทฤษฎี : ภาษา Go เป็นภาษาโปรแกรมคอมพิวเตอร์ที่พัฒนาโดย Google มีจุดเด่นด้านประสิทธิภาพการทำงานที่รวดเร็ว (Compiled Language) มีความปลอดภัยสูง และรองรับการทำงานแบบคู่ขนาน (Concurrency) ได้อย่างมีประสิทธิภาพ สำหรับ Fiber นั้นเป็นเว็บเฟรมเวิร์ก (Web Framework) ที่สร้างขึ้นบนภาษา Go ซึ่งได้รับแรงบันดาลใจจาก Express.js ออกแบบมาเพื่อการพัฒนา API ที่รวดเร็วและใช้ทรัพยากรหน่วยความจำต่ำ

การนำมาประยุกต์ใช้ : ใช้ในการพัฒนาระบบหลังบ้าน (Backend Server) ทำหน้าที่เป็นศูนย์กลาง (Orchestrator) เพื่อรับคำสั่งจากหน้าเว็บ ควบคุมการอ่านไฟล์ตั้งค่า (.env และ

test_config.json) และเป็นตัวสั่งการ (Process Spawn) ให้ระบบบอททดสอบทำงาน รวมถึงจัดการวงจรชีวิตของโปรเซส (Process Lifecycle) เช่น การสั่งยกเลิกการทดสอบกลางคัน เป็นต้น

**2.2.3 ไลบรารี React และการพัฒนาส่วนติดต่อผู้ใช้งาน (Frontend Development)**

ทฤษฎี : React คือไลบรารีจาวาสคริปต์ (JavaScript Library) ที่พัฒนาโดย Facebook สำหรับสร้างส่วนติดต่อผู้ใช้งาน (User Interface) โดยมีโครงสร้างพื้นฐานแบบ Component-Based Architecture ที่แบ่งหน้าเว็บออกเป็นส่วนย่อยๆ ทำให้สามารถนำโค้ดกลับมาใช้ซ้ำ (Reusable) ได้ และมีระบบ Virtual DOM ที่ช่วยให้การอัปเดตหน้าจอและการแสดงผลข้อมูลมีความรวดเร็วและราบรื่น

การนำมาประยุกต์ใช้ : นำมาพัฒนาส่วนหน้าบ้านของระบบ เพื่อสร้าง Dashboard สำหรับแสดงรายชื่อระบบที่ต้องการทดสอบ โดยใช้ไลบรารี Mantine React Table ในการสร้างตารางแสดงผลข้อมูล (Data Grid) ที่มีประสิทธิภาพสูง รองรับการค้นหาและแบ่งหน้าเว็บโดยอัตโนมัติ และสร้างหน้าต่างแสดงประวัติการทดสอบ (History View) ที่ผู้ใช้งานสามารถโต้ตอบ สั่งรันชุดทดสอบ และดูภาพหลักฐาน (Screenshots) ได้อย่างเป็นมิตรต่อผู้ใช้ (User-friendly)

**2.2.4 เครื่องมือ Playwright และ Node.js**

ทฤษฎี : Node.js เป็นสภาพแวดล้อมสำหรับการรันโค้ดจาวาสคริปต์ทางฝั่งเซิร์ฟเวอร์ (Runtime Environment) ส่วน Playwright เป็นเฟรมเวิร์กโอเพนซอร์สที่พัฒนาโดย Microsoft สำหรับการทดสอบเว็บแอปพลิเคชันแบบ End-to-End รองรับการทำงานข้ามเบราว์เซอร์ (Chromium, Firefox, WebKit) มีความสามารถในการจำลองพฤติกรรมมนุษย์ เช่น การคลิก การพิมพ์อักษร การรอสถานะโหลด (Auto-wait) และการดักจับข้อมูลเครือข่าย (Network Interception)

การนำมาประยุกต์ใช้ : ใช้เป็นแกนหลักในส่วนปฏิบัติการทดสอบ (Automation Engine) โดย Playwright จะทำงานบน Node.js ทำหน้าที่อ่านสคริปต์ (Scenario) แล้วเปิดเบราว์เซอร์ขึ้นมาจำลองการทำงานของระบบปลายทาง จับภาพหน้าจอในแต่ละขั้นตอน และสรุปผลสถานะผ่าน/ไม่ผ่าน (Pass/Fail) ส่งกลับไปยัง Backend

**2.2.5 โครงสร้างข้อมูล JSON และสถาปัตยกรรม REST API**

ทฤษฎี : JSON (JavaScript Object Notation) เป็นรูปแบบการจัดเก็บและแลกเปลี่ยนข้อมูลที่มีน้ำหนักเบาและมนุษย์สามารถอ่านทำความเข้าใจได้ง่าย ส่วน REST API (Representational State Transfer) เป็นสถาปัตยกรรมการออกแบบเว็บเซอร์วิสเพื่อสื่อสารระหว่างไคลเอนต์และเซิร์ฟเวอร์ผ่านโปรโตคอล HTTP

การนำมาประยุกต์ใช้ : โครงงานนี้ใช้ JSON เป็นโครงสร้างหลักในการเก็บรายชื่อเทสเคส (test_config.json) และขั้นตอนการทดสอบ (Scenario Scripts) รวมถึงใช้ REST API เป็นช่องทางสื่อสารหลักระหว่างระบบหน้าบ้าน (React) และระบบหลังบ้าน (Go) เพื่อส่งคำสั่งและรับผลลัพธ์การทดสอบกลับมาแสดงผล

# บทที่ 3
# กระบวนการปฏิบัติงานสหกิจศึกษา

การปฏิบัติงานสหกิจศึกษา ณ มูลนิธิแม่ฟ้าหลวง ในพระบรมราชูปถัมภ์ ผู้จัดทำได้รับมอบหมายให้พัฒนาโครงงาน "การพัฒนาเว็บแอปพลิเคชันเพื่อบริหารจัดการและดำเนินการทดสอบซอฟต์แวร์อัตโนมัติ" โดยมีกระบวนการและขั้นตอนการปฏิบัติงานที่เป็นระบบ ดังรายละเอียดต่อไปนี้

## 3.1 ขั้นตอนการปฏิบัติงาน (โครงงาน/งานวิจัย)

ผู้จัดทำได้นำวงจรการพัฒนาซอฟต์แวร์ (Software Development Life Cycle : SDLC) มาประยุกต์ใช้ในการดำเนินงาน โดยแบ่งขั้นตอนออกเป็น 4 ระยะ ดังนี้

3.1.1 การศึกษาความต้องการและออกแบบสถาปัตยกรรมระบบ (System Architecture Design)

ร่วมประชุมกับพี่เลี้ยงและทีมพัฒนาซอฟต์แวร์ของมูลนิธิ เพื่อวิเคราะห์ปัญหาของการทดสอบระบบแบบเดิม (Manual Testing)

รวบรวมความต้องการของระบบ (Requirement Gathering) เช่น ต้องการให้มีหน้าเว็บสั่งรันง่ายๆ และต้องเก็บรูปภาพหลักฐาน (Screenshots) แยกตามรอบการทำงานได้

ออกแบบสถาปัตยกรรมระบบ (System Architecture) ให้ทำงานประสานกัน 3 ส่วน คือ React (หน้าบ้าน), Go (หลังบ้าน) และ Playwright (บอทจำลองการทดสอบ)

3.1.2 การพัฒนาระบบส่วนหน้าบ้านและหลังบ้าน (Frontend & Backend Development)

ส่วนหลังบ้าน (Backend) : พัฒนา API ด้วยภาษา Go โดยใช้ Fiber Framework เพื่อรับคำสั่งจากหน้าเว็บ สร้างฟังก์ชันอ่านค่าตัวแปรความลับจากไฟล์ .env และพัฒนาระบบจัดการโปรเซส (Process Management) สำหรับสั่งรันหรือยกเลิกการทำงานของบอททดสอบ

ส่วนหน้าบ้าน (Frontend) : พัฒนาด้วย React **และไลบรารี Mantine React Table** สร้างส่วนติดต่อผู้ใช้งาน (UI) ที่มีตารางแสดงชุดการทดสอบ (Test Suites) ปุ่มสั่งรัน และหน้าต่างแสดงประวัติพร้อมหลักฐานการทดสอบ (History View)

3.1.3 การพัฒนาสคริปต์ทดสอบอัตโนมัติ (Automation Scripting)

เขียนโปรแกรมด้วย Node.js และ Playwright เพื่อจำลองพฤติกรรมการคลิก การพิมพ์ และการนำทางบนเบราว์เซอร์

ออกแบบโครงสร้างข้อมูลแบบ JSON (Data-Driven Configuration) เพื่อใช้เป็นตัวกำหนดขั้นตอนการทดสอบ (Scenario) ทำให้ระบบสามารถอ่านค่าและรันการทดสอบตามข้อมูลที่ตั้งไว้ได้แบบไดนามิก

3.1.4 การทดสอบและปรับปรุงการทำงานของระบบ (Testing and Deployment)

ดำเนินการทดสอบการทำงานของเครื่องมือ (Tool Testing) เช่น ทดสอบการส่งคำสั่งรันพร้อมกัน ทดสอบระบบจัดการความผิดพลาด (Error Handling) และทดสอบการสั่งยกเลิก (Kill Process) กรณีที่ระบบค้าง เพื่อให้มั่นใจว่าเครื่องมือทำงานได้อย่างมีเสถียรภาพก่อนนำไปใช้งานจริง

**3.2 การดำเนินงานมีความถูกต้อง มีระเบียบแบบแผน และทำให้นักศึกษามีโอกาสใช้วิชาความรู้/ทักษะตามที่เรียนมา**

การปฏิบัติงานในโครงงานนี้ ผู้จัดทำได้นำองค์ความรู้และทักษะทางวิชาการจากสาขาวิชาที่ได้ศึกษาในระดับมหาวิทยาลัย มาประยุกต์ใช้ในการแก้ปัญหาจริงอย่างเป็นระบบและต่อยอดสู่การเรียนรู้เทคโนโลยีใหม่ๆ ได้แก่

**กลุ่มความรู้ด้านวิศวกรรมซอฟต์แวร์และการออกแบบระบบ (Software Engineering & System Design)** นำความรู้เรื่องวงจรการพัฒนาซอฟต์แวร์ (SDLC) การรวบรวมความต้องการ และการออกแบบสถาปัตยกรรมระบบแบบแยกส่วน (Microservices Concept) มาใช้ในการวางโครงสร้างโปรเจกต์

**กลุ่มความรู้ด้านการพัฒนาเว็บแอปพลิเคชัน (Web Application Development)** นำความรู้พื้นฐานเกี่ยวกับการทำงานของโครงสร้างหน้าเว็บ (HTML, CSS และ JavaScript) มาเป็นฐานคิดสำคัญในการทำความเข้าใจการทำงานของเบราว์เซอร์ ซึ่งช่วยให้ผู้จัดทำสามารถต่อยอดและเรียนรู้เทคโนโลยีใหม่ที่ซับซ้อนขึ้นอย่างไลบรารี React รวมถึงการสร้าง API ด้วยภาษา Go ในช่วงฝึกงานได้อย่างรวดเร็ว

**กลุ่มความรู้ด้านระบบปฏิบัติการ (Operating Systems)** นำทักษะและความเข้าใจเรื่องการจัดการโปรเซส (Process Management) พื้นฐาน มาประยุกต์ใช้ในการเขียนโค้ดเพื่อควบคุมการสร้างโปรเซส (Spawn Process) ของ Node.js รวมถึงการจัดการหน่วยความจำและการสั่งปิดโปรเซส (Task Kill) อย่างปลอดภัย

**3.3 ความพึงพอใจของสถานประกอบการต่อโครงงาน**

จากการนำระบบไปให้ทีมพัฒนาซอฟต์แวร์ของมูลนิธิได้ทดลองใช้งานจริง พบว่าผู้ใช้งานมีความพึงพอใจในระดับดี โดยได้รับข้อเสนอแนะเชิงบวก (Feedback) ในด้านต่างๆ ดังนี้

1. ความง่ายในการใช้งาน (User-Friendly) ผู้ใช้งานชื่นชอบที่ระบบสามารถกดสั่งรันการทดสอบได้ผ่านหน้าเว็บโดยไม่ต้องเขียนคำสั่ง (Command Line) เอง

2. การประหยัดเวลา (Time-saving) ระบบช่วยลดภาระงานของบุคลากรได้อย่างชัดเจน จากเดิมที่ต้องใช้เวลาทดสอบหลายนาทีเหลือเพียงไม่กี่วินาที

3. ความสมบูรณ์ของหลักฐาน (Evidence Completeness) โฟลเดอร์การจัดเก็บหลักฐาน (Artifacts) มีการจัดระเบียบตามรอบการทำงาน (Run-ID) ได้อย่างดีเยี่ยม ทำให้ผู้ตรวจประเมิน

4. สามารถตรวจสอบย้อนหลังได้ง่ายและลดปัญหาไฟล์สูญหาย

3.4 เป็นงานที่นำไปใช้ประโยชน์ได้อย่างเป็นรูปธรรมในสถานประกอบการ

โครงงานนี้ก่อให้เกิดประโยชน์อย่างเป็นรูปธรรมต่อมูลนิธิแม่ฟ้าหลวงฯ ทั้งในระยะสั้นและระยะยาว ได้แก่

1. ยกระดับคุณภาพของซอฟต์แวร์ (Quality Assurance) มูลนิธิมีเครื่องมือทดสอบซอฟต์แวร์ที่มีความแม่นยำสูง ปราศจากความผิดพลาดที่เกิดจากความเหนื่อยล้าของมนุษย์ (Human Error) 2. ทำให้ระบบซอฟต์แวร์ภายใน (Internal Systems) มีความเสถียรก่อนนำไปใช้งานจริง

3. ลดต้นทุนด้านเวลาและทรัพยากรบุคคล (Cost & Time Reduction) พนักงานและนักพัฒนา

4. ซอฟต์แวร์สามารถนำเวลาที่เคยเสียไปกับการทดสอบด้วยตนเอง ไปใช้ในการพัฒนาระบบอื่นๆ ที่สร้างมูลค่าเพิ่มให้กับองค์กรได้มากขึ้น

5. รองรับการขยายตัวในอนาคต (High Scalability) ด้วยการออกแบบระบบแบบ Configuration-Driven ทำให้มูลนิธิสามารถนำแพลตฟอร์มนี้ไปใช้ทดสอบระบบแอปพลิเคชันตัวอื่นๆ ในอนาคตได้ทันที เพียงแค่เพิ่มไฟล์ตั้งค่า JSON โดยไม่ต้องรื้อโครงสร้างโปรแกรมใหม่ ถือเป็นการลงทุนด้านเทคโนโลยีที่คุ้มค่าและยั่งยืน

# บทที่ 4
# ผลการปฏิบัติงาน

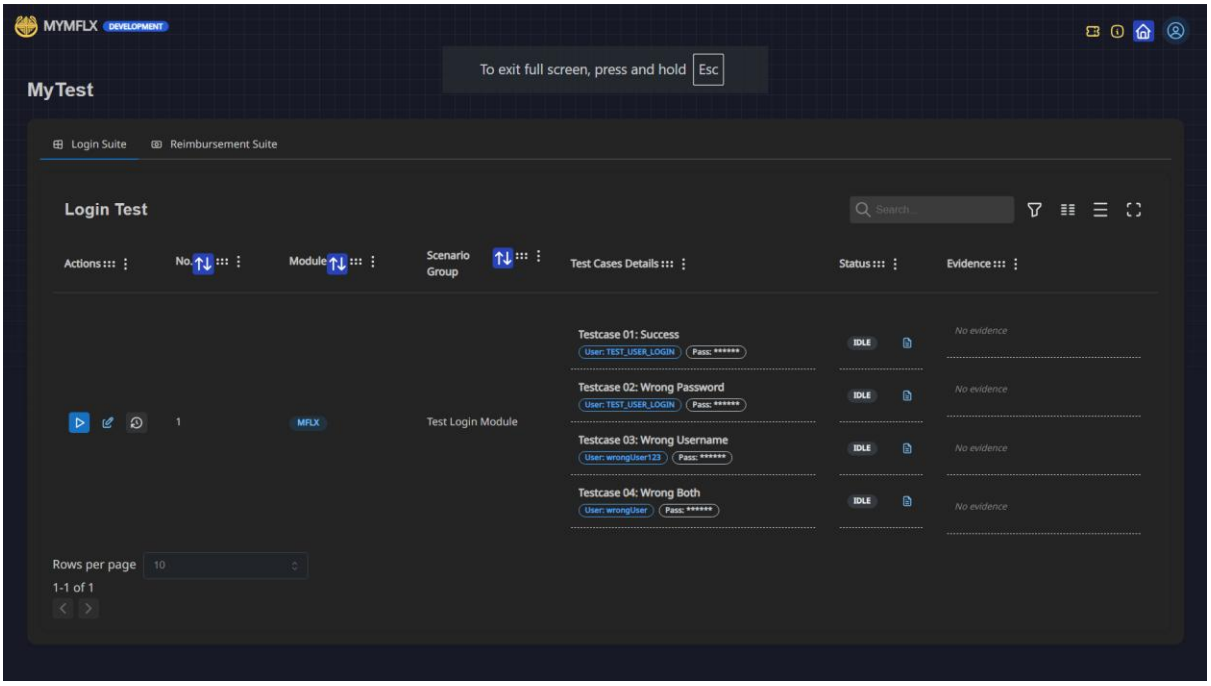
จากการปฏิบัติงานสหกิจศึกษา ณ มูลนิธิแม่ฟ้าหลวง ในพระบรมราชูปถัมภ์ ในโครงงาน "การพัฒนาเว็บแอปพลิเคชันเพื่อบริหารจัดการและดำเนินการทดสอบซอฟต์แวร์อัตโนมัติ" ผู้จัดทำได้ดำเนินการออกแบบ พัฒนา และทดสอบระบบจนเสร็จสมบูรณ์ตามวัตถุประสงค์ที่ตั้งไว้ โดยสามารถแบ่งผลการปฏิบัติงานออกเป็น 4 ส่วนหลัก ดังรายละเอียดต่อไปนี้

## 4.1 ผลการพัฒนาระบบส่วนติดต่อผู้ใช้งาน (Frontend - React)

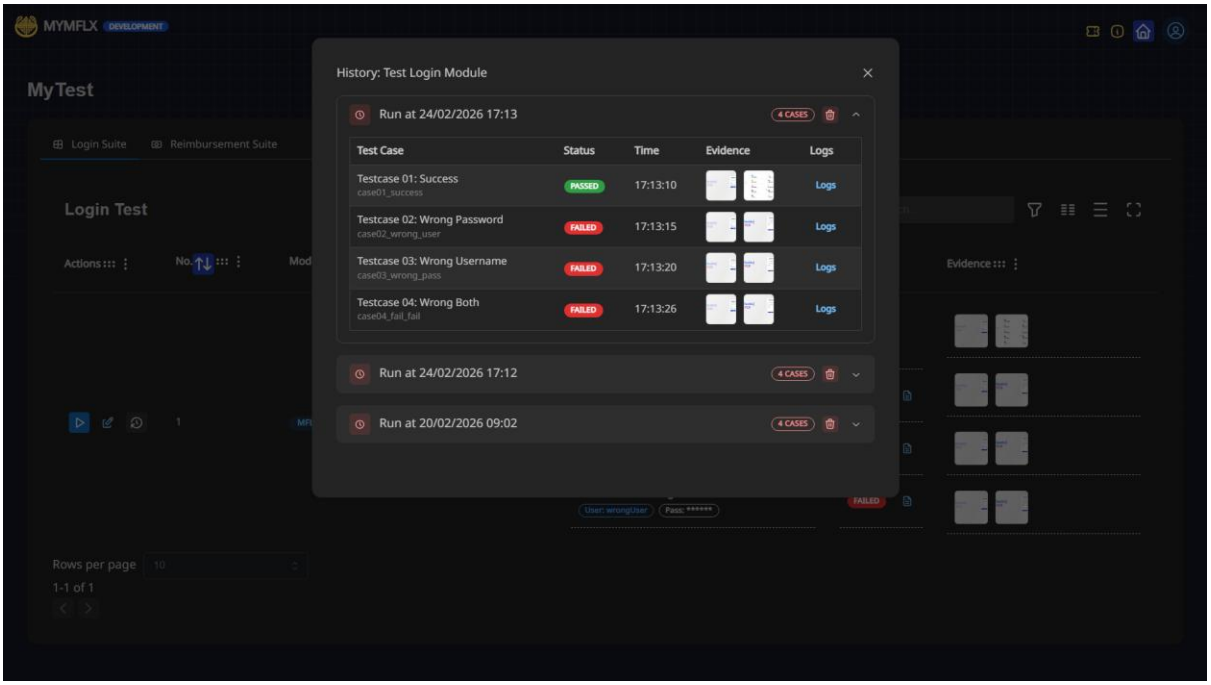
ผลลัพธ์จากการพัฒนาส่วนหน้าบ้านด้วยไลบรารี React ได้เว็บแอปพลิเคชันที่ทำหน้าที่เป็นศูนย์กลางให้ผู้ใช้งานสามารถโต้ตอบกับระบบได้อย่างสะดวก (User Interface) โดยระบบถูกออกแบบให้รองรับการแสดงผลหลายโครงการพร้อมกัน ซึ่งในปัจจุบันได้นำร่องใช้งานกับ 2 ระบบหลักของมูลนิธิ ได้แก่ ระบบเข้าสู่ระบบ (Login System) และ ระบบเบิกจ่าย (My Reimbursement) โดยมีผลการทำงานดังนี้

**หน้ากระดานข้อมูลรวม (Dashboard / Test Suites View)** ระบบสามารถอ่านค่าจากไฟล์การตั้งค่าและแสดงผลรายการชุดการทดสอบทั้งระบบ Login และระบบ My Reimbursement ขึ้นมาบนหน้าจอหลักได้อย่างถูกต้อง โดยมีการนำ Mantine React Table มาช่วยจัดรูปแบบตารางให้รองรับการค้นหาข้อมูล (Search) และการแบ่งหน้า (Pagination) ทำให้ผู้ใช้งานสามารถดูรายละเอียด และสามารถสั่งดำเนินการทดสอบเฉพาะระบบที่ต้องการได้ทันทีผ่านการคลิกปุ่ม "Run Test"

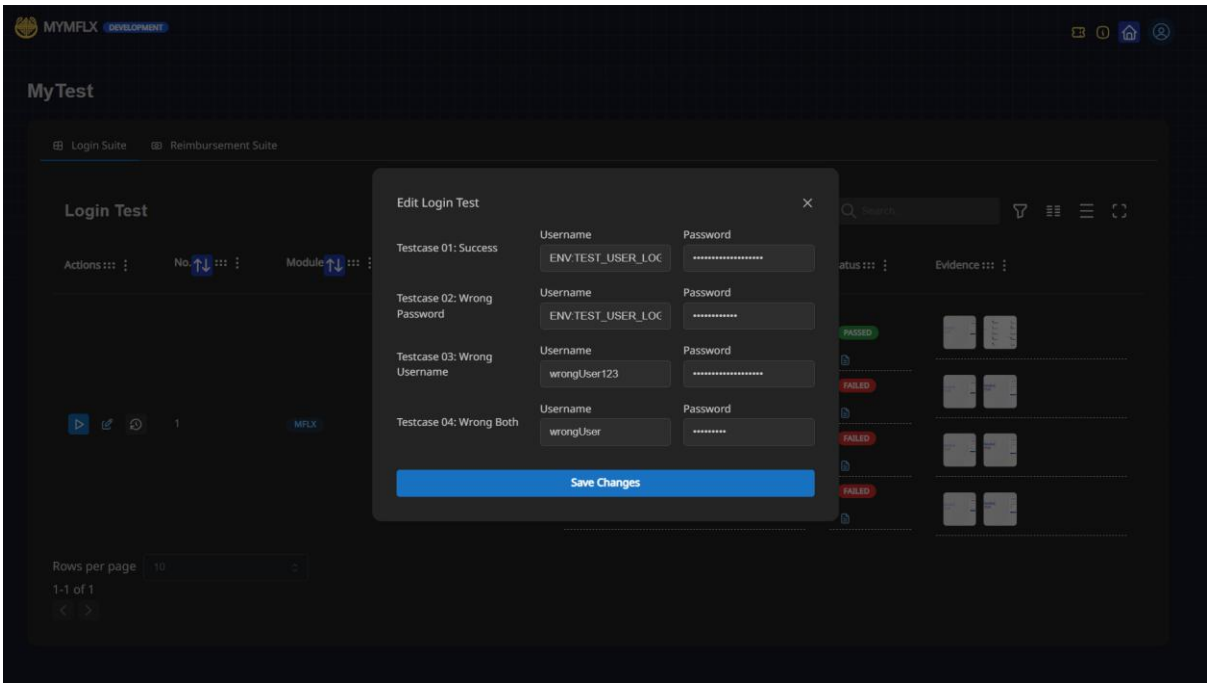
**หน้าต่างประวัติการทดสอบ (History View)** เมื่อการทดสอบเสร็จสิ้น ระบบจะแสดงประวัติการรันในอดีต พร้อมบอกสถานะว่าการทดสอบในรอบนั้นๆ สำเร็จ (Pass) หรือล้มเหลว (Fail) รวมถึงสามารถกดเข้าไปดูภาพถ่ายหน้าจอ (Screenshots) ที่ระบบบันทึกไว้ได้อย่างรวดเร็ว



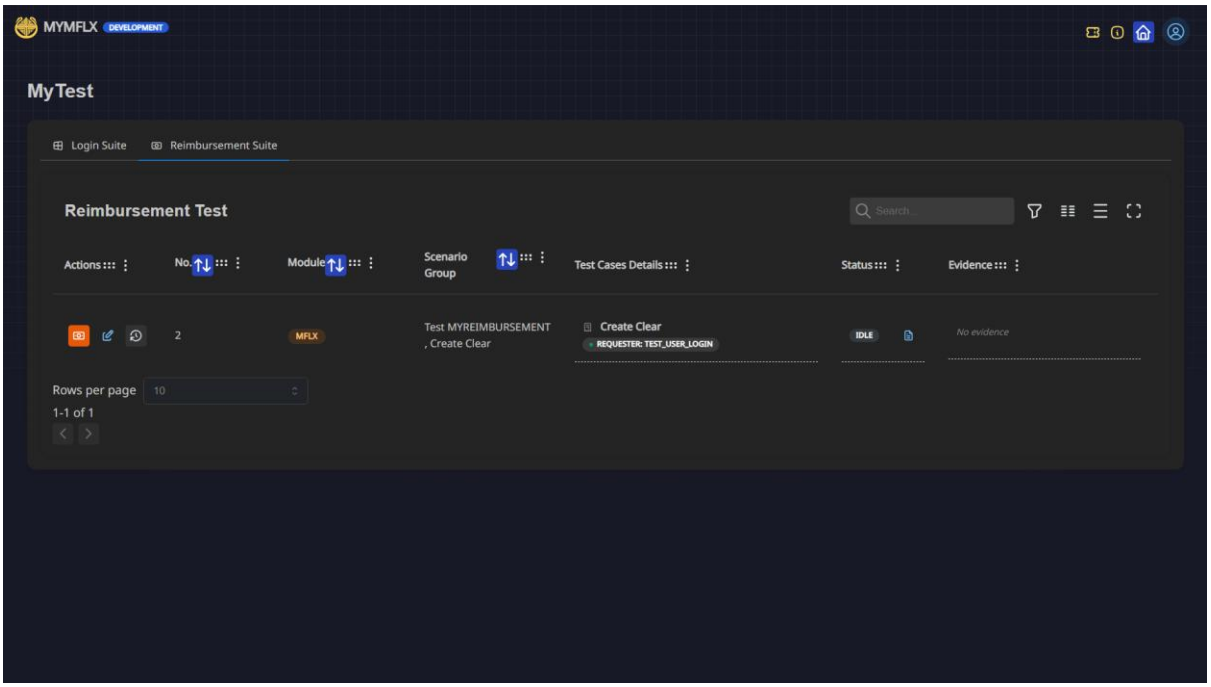

รูปที่ 4.1 หน้าจอหลักแสดงรายการชุดการทดสอบของระบบเข้าสู่ระบบ



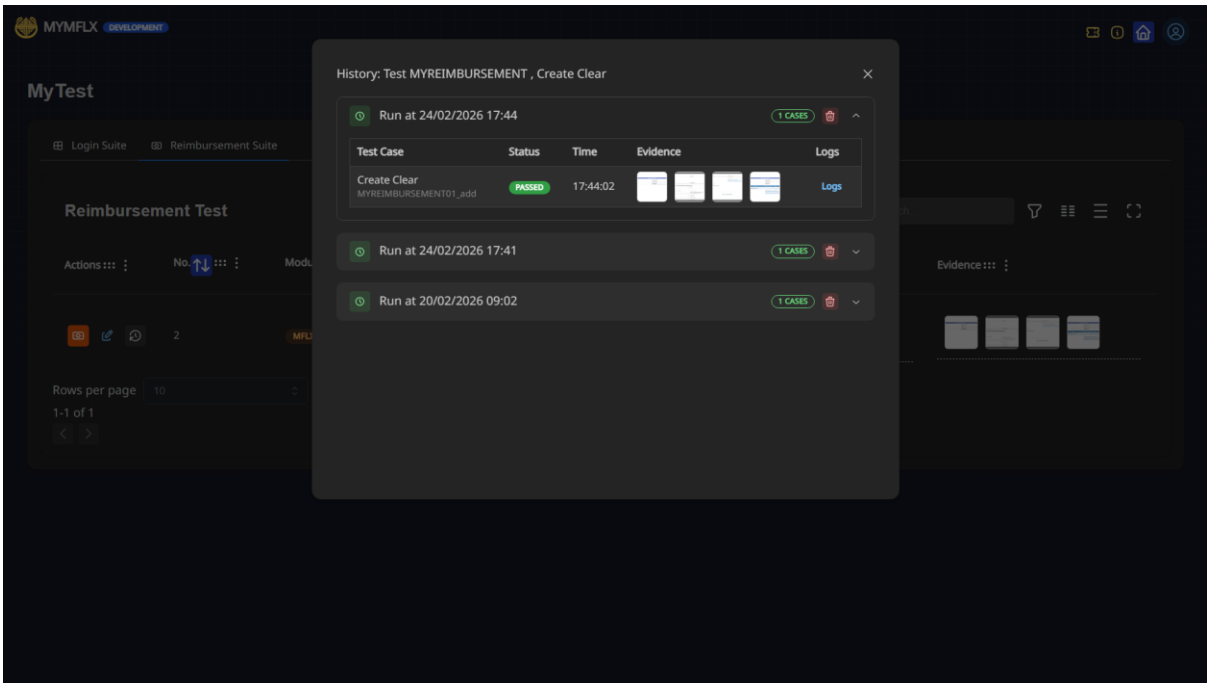

รูปที่ 4.2 หน้าจอหลักแสดงรายการประวัติชุดการทดสอบของระบบเข้าสู่ระบบ



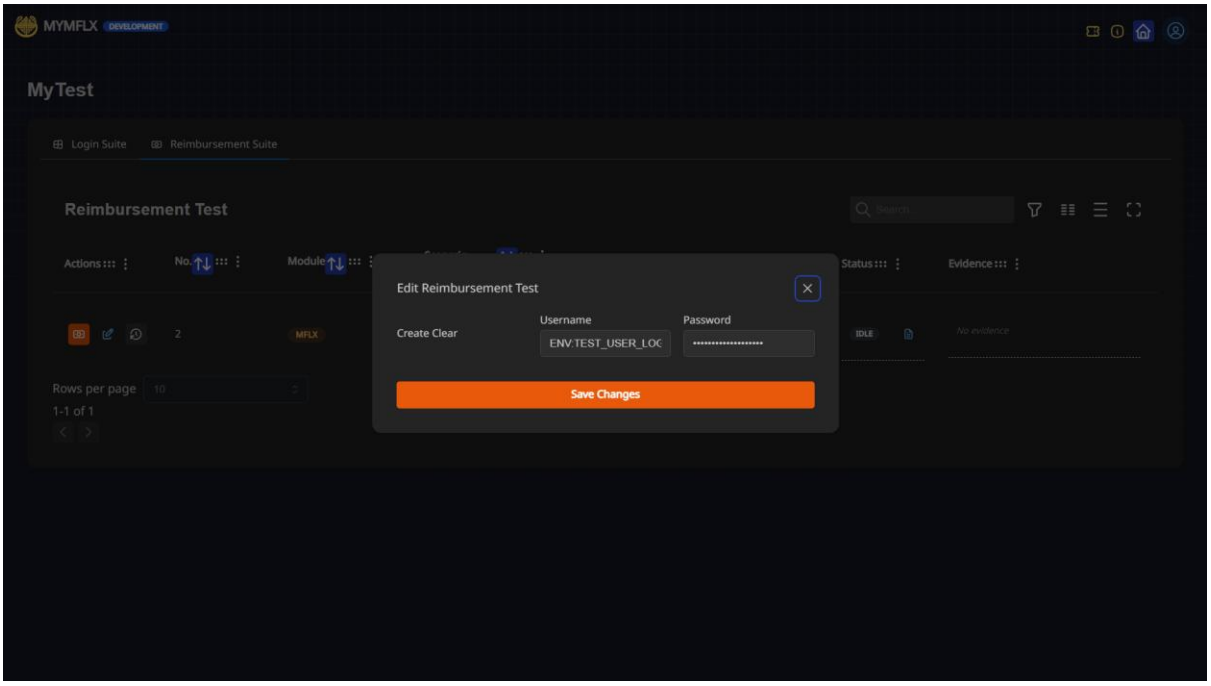

รูปที่ 4.3 หน้าจอหลักแสดงรายการแก้ไข user password ในกรณีที่ต้องการแก้ไขการ test เอง ของระบบเข้าสู่ระบบ



รูปที่ 4.4 หน้าจอหลักแสดงรายการชุดการทดสอบของระบบขออนุมัติสั่งซื้อ




รูปที่ 4.5 หน้าจอหลักแสดงรายการประวัติชุดการทดสอบของระบบขออนุมัติสั่งซื้อ



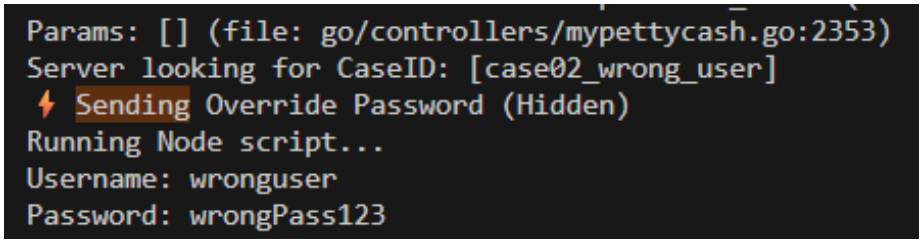

รูปที่ 4.6 หน้าจอหลักแสดงรายการแก้ไข user password ในกรณีที่ต้องการแก้ไขการ test เอง ของระบบขออนุมัติสั่งซื้อ

**4.2 ผลการพัฒนาระบบจัดการหลังบ้าน (Backend - Go & Fiber)**

ผลลัพธ์จากการพัฒนาระบบหลังบ้านด้วยภาษา Go และ Fiber Framework ได้ระบบ API (Application Programming Interface) ที่มีความเสถียรและสามารถจัดการคำสั่งที่ซับซ้อนได้ โดยมีผลการทำงานดังนี้

**ระบบศูนย์กลางควบคุม (Orchestrator API)** ระบบสามารถรับคำสั่งสั่งรันการทดสอบจากหน้าเว็บ (Frontend) และทำการสร้างรหัสอ้างอิงรอบการทำงาน (Run ID) เพื่อใช้ในการติดตามสถานะ

**การจัดการโปรเซส (Process Management)** ระบบสามารถเรียกใช้งาน (Spawn Process) ตัวบอททดสอบที่เขียนด้วย Node.js ได้อย่างแม่นยำ และสามารถส่งค่าตัวแปรความลับ (Environment Variables) เช่น ชื่อผู้ใช้งานและรหัสผ่านเข้าสู่กระบวนการทดสอบได้อย่างปลอดภัย นอกจากนี้ยังมีระบบสั่งยกเลิกการทำงาน (Kill Process) กรณีที่ผู้ใช้ต้องการหยุดการทดสอบกลางคัน




รูปที่ 4.7 ตัวอย่างบันทึกการทำงาน (Log) ของระบบหลังบ้านขณะส่งค่าความลับและเรียกใช้งานบอททดสอบ

**4.3 ผลการพัฒนาระบบปฏิบัติการทดสอบอัตโนมัติ (Automation Engine - Playwright)**

ผลลัพธ์จากการพัฒนาเครื่องมือทดสอบด้วย Node.js และ Playwright ได้ระบบบอทอัตโนมัติ (Automation Bot) ที่สามารถจำลองพฤติกรรมการใช้งานบนเว็บเบราว์เซอร์ได้อย่างสมบูรณ์ โดยมีผลการทำงานดังนี้

**การอ่านค่าและดำเนินการทดสอบ (Data-Driven Execution)** ระบบสามารถอ่านขั้นตอนการทดสอบจากไฟล์ JSON (Scenario Script) และจำลองการคลิก การกรอกข้อความ และการรอโหลดหน้าเว็บ (Auto-wait) ตามที่กำหนดไว้ได้อย่างถูกต้องแม่นยำ

**การจัดการข้อผิดพลาด (Error Handling):** ในกรณีที่หน้าเว็บปลายทางเกิดข้อผิดพลาด หรือบอทไม่สามารถค้นหาปุ่มกดได้ ระบบจะทำการหยุดการทำงานในขั้นตอนนี้ บันทึกสาเหตุของข้อผิดพลาด (Log) และแจ้งสถานะกลับไปยังหลังบ้านโดยที่ระบบเซิร์ฟเวอร์หลักไม่ล่ม (Crash)



```
{
  "testCase": {
    "caseId": "case02_wrong_user",
    "steps": [
      {
        "action": "navigate",
        "url": "{{BASE_URL}}/login"
      },
      {
        "action": "fill_login_form",
        "username": "ENV:TEST_USER_LOGIN",
        "password": "ENV:TEST_PASS_LOGIN",
        "description": "1. กรอกรหัสลงในฟอร์ม"
      },
      {
        "action": "screenshot",
        "name": "after_fill_data",
        "description": "2. Screenshot หลังกรอกรหัสเสร็จ"
      },
      { "action": "waitForTimeout", "ms": 1000 },
      {
        "action": "click_login_button",
        "description": "3. กดปุ่ม Login"
      },
      {
        "action": "screenshot",
        "name": "after_login_clicked",
        "description": "4. Screenshot หลังกดปุ่ม Login"
      },
      { "action": "waitForTimeout", "ms": 2000 }
    ]
  }
}
```

รูปที่ 4.8 ตัวอย่างโครงสร้างไฟล์การตั้งค่าชุดทดสอบ test login (JSON Configuration)

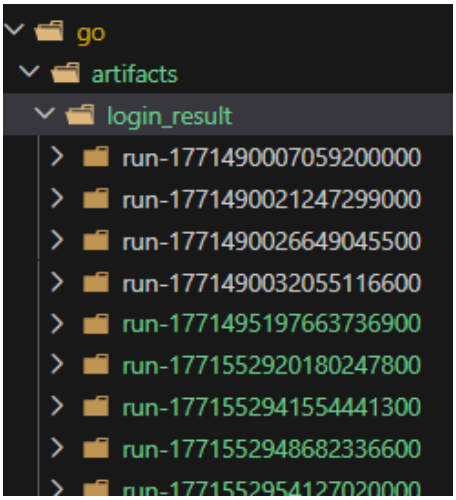
รูปที่ 4.9 หน้าจอขณะระบบปฏิบัติการทดสอบกำลังจำลองการทำงานบนเว็บเบราว์เซอร์

## 4.4 ผลลัพธ์การจัดเก็บหลักฐานการทดสอบ (Testing Artifacts & Logs)

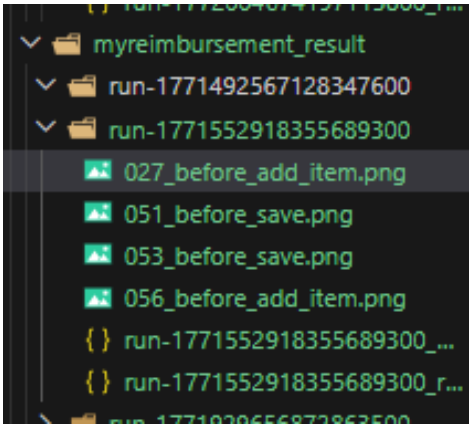
ผลลัพธ์ที่สำคัญที่สุดของโครงงานนี้คือ ระบบสามารถจัดการโฟลเดอร์และไฟล์หลักฐานได้อย่างเป็นระเบียบโดยอัตโนมัติ (Automated Artifacts Collection) โดยมีผลการทำงานดังนี้

**การสร้างโฟลเดอร์แยกตามรอบ (Run-ID Separation)** ทุกครั้งที่มีการสั่งรัน ระบบจะสร้างโฟลเดอร์ใหม่ตามหมายเลข Run ID และจัดเก็บภาพถ่ายหน้าจอ (Screenshots) ในแต่ละขั้นตอนลงในโฟลเดอร์นั้นๆ อย่างเป็นระเบียบ

**การสร้างไฟล์สรุปผล (Result Output)** เมื่อจบการทำงาน ระบบจะสร้างไฟล์ result.json ซึ่งสรุปภาพรวมว่าในรอบการทำงานนี้มีกี่ขั้นตอนที่ผ่าน กี่ขั้นตอนที่ล้มเหลว ใช้เวลาทำงานทั้งหมดกี่วินาที เพื่อส่งข้อมูลกลับไปแสดงผลที่หน้าจอเว็บแอปพลิเคชันต่อไป




รูปที่ 4.10 โครงสร้างโฟลเดอร์จัดเก็บหลักฐานที่แบ่งตามหมายเลขอ้างอิง (Run ID)




รูปที่ 4.11 ตัวอย่างภาพถ่ายหน้าจอและไฟล์สรุปผลที่ระบบจัดเก็บไว้โดยอัตโนมัติ

# บทที่ 5
# สรุปผลการปฏิบัติงานสหกิจศึกษา

จากการที่ผู้จัดทำได้เข้าไปปฏิบัติงานสหกิจศึกษา ณ มูลนิธิแม่ฟ้าหลวง ในพระบรมราชูปถัมภ์ ในตำแหน่ง นักศึกษาสหกิจศึกษา ตำแหน่งนักพัฒนาซอฟต์แวร์ (Full-Stack Developer) ตั้งแต่วันที่ 3 พฤศจิกายน พ.ศ.2568 ถึงวันที่ 20 กุมภาพันธ์ พ.ศ.2569 ผู้จัดทำได้รับมอบหมายให้พัฒนาโครงงาน "การพัฒนาเว็บแอปพลิเคชันเพื่อบริหารจัดการและดำเนินการทดสอบซอฟต์แวร์อัตโนมัติ" ซึ่งสามารถสรุปผลการปฏิบัติงานและรายละเอียดต่างๆ ได้ดังนี้

## 5.1 สรุปผลโครงงานสหกิจศึกษา

การพัฒนาเว็บแอปพลิเคชันเพื่อบริหารจัดการและดำเนินการทดสอบซอฟต์แวร์อัตโนมัติ ประสบความสำเร็จตามวัตถุประสงค์ที่ตั้งไว้ทุกประการ ระบบสามารถทำงานร่วมกันได้อย่างสมบูรณ์แบบทั้ง 3 ส่วน ได้แก่ ส่วนหน้าบ้าน (React) ที่มีส่วนติดต่อผู้ใช้งานที่ใช้งานง่าย, ส่วนหลังบ้าน (Go & Fiber) ที่ทำหน้าที่เป็นศูนย์กลางควบคุม API อย่างมีประสิทธิภาพ และส่วนปฏิบัติการทดสอบ (Playwright) ที่สามารถจำลองการทำงานบนเบราว์เซอร์ได้อย่างแม่นยำ ผลลัพธ์ที่ได้คือ ระบบสามารถช่วยมูลนิธิลดระยะเวลาในการทดสอบซอฟต์แวร์แบบเดิม (Manual Testing) ลงได้อย่างมาก ลดความผิดพลาดจากตัวบุคคล (Human Error) และมีระบบจัดการจัดเก็บหลักฐานการทดสอบ (Artifacts & Screenshots) ตามหมายเลขรอบการรัน (Run ID) โดยอัตโนมัติ ซึ่งช่วยยกระดับมาตรฐานการทดสอบซอฟต์แวร์ขององค์กรให้มีความรวดเร็วและเป็นระบบมากยิ่งขึ้น

## 5.2 สิ่งที่ได้รับจากการปฏิบัติงานสหกิจศึกษา

ประสบการณ์การทำงานจริง ได้เรียนรู้ระบบการทำงานขององค์กรขนาดใหญ่อย่างมูลนิธิแม่ฟ้าหลวงฯ การทำงานร่วมกับทีมพัฒนาซอฟต์แวร์ (Dev Team) และการสื่อสารอย่างเป็นมืออาชีพ ความเข้าใจในวงจรการพัฒนาซอฟต์แวร์ (SDLC) ได้เห็นภาพรวมตั้งแต่การรับโจทย์ความต้องการ (Requirement) จากผู้ใช้งานจริง การออกแบบสถาปัตยกรรมระบบ การลงมือเขียนโค้ด ไปจนถึงการทดสอบและนำไปใช้งานจริง

การปรับตัวและความรับผิดชอบ ได้ฝึกฝนความมีระเบียบวินัย การจัดการเวลาในการทำงานให้สำเร็จตามเป้าหมาย (Sprint/Timeline) ที่ได้รับมอบหมาย และความรับผิดชอบต่องานที่ส่งผลกระทบต่อองค์กร

### 5.3 ปัญหาและอุปสรรคที่พบจากการปฏิบัติงาน

ระหว่างการปฏิบัติงานและการพัฒนาระบบ ผู้จัดทำพบปัญหาและอุปสรรคบางประการ ดังนี้ การเรียนรู้เทคโนโลยีใหม่ในเวลาจำกัด โครงงานนี้มีการใช้ภาษา Go (Fiber Framework) และ Playwright ซึ่งเป็นเทคโนโลยีที่ผู้จัดทำยังไม่คุ้นเคยมากนัก ทำให้ในช่วงแรกต้องใช้เวลาในการศึกษาโครงสร้างภาษาและคู่มือการใช้งาน (Documentation) ค่อนข้างมาก

การจัดการการทำงานแบบคู่ขนาน (Asynchronous Processing) การเขียนให้ระบบหลังบ้าน (Go) สั่งรันบอททดสอบ (Node.js) และต้องรอรับผลลัพธ์กลับมาแสดงที่หน้าเว็บ โดยไม่ทำให้เซิร์ฟเวอร์ค้าง (Block) เป็นเรื่องที่ท้าทาย

ความไม่แน่นอนของหน้าเว็บปลายทาง (Dynamic Web Elements) ในการเขียนสคริปต์ Playwright บางครั้งหน้าเว็บปลายทางโหลดข้อมูลช้า หรือมีองค์ประกอบ (Elements) เปลี่ยนแปลงตำแหน่ง ทำให้บอทหาปุ่มกดไม่เจอและเกิดข้อผิดพลาด (Timeout Error)

### 5.4 วิธีแก้ไขปัญหาที่พบจากการปฏิบัติงาน

การแก้ปัญหาด้านการเรียนรู้ ผู้จัดทำได้ขอคำปรึกษาจากพี่เลี้ยง (Mentor) อย่างสม่ำเสมอ และใช้เวลานอกเวลางานในการศึกษาเอกสารคู่มืออย่างเป็นทางการ (Official Documentation) ของ Go และ Playwright รวมถึงการดูตัวอย่างโค้ด (Best Practices) จากแหล่งข้อมูลที่น่าเชื่อถือ

การแก้ปัญหาการทำงานแบบคู่ขนาน ศึกษาและนำเทคนิคการจัดการ Process และ Goroutines ในภาษา Go มาประยุกต์ใช้ ทำให้สามารถสั่งรันบอทใน Background Process ได้อย่างปลอดภัย และสร้างระบบ API เพื่อให้หน้าบ้านสามารถดึงสถานะ (Polling) ไปแสดงผลได้

การแก้ปัญหาหน้าเว็บปลายทาง ได้ปรับปรุงสคริปต์ทดสอบโดยเพิ่มฟังก์ชันการรออัตโนมัติ (Auto-wait) และการดักจับข้อผิดพลาด (Try-Catch / Error Handling) เพื่อให้บอทรอจนกว่าหน้าเว็บจะโหลดเสร็จสมบูรณ์ก่อนทำงานขั้นต่อไป ช่วยให้การทดสอบมีความเสถียรมากขึ้น

### 5.5 ความรู้และทักษะใหม่ที่ได้รับระหว่างปฏิบัติงานสหกิจศึกษา

ทักษะด้านภาษา Go (Golang) ได้รับความรู้ในการเขียนภาษา Go และการใช้ Fiber Framework เพื่อสร้าง RESTful API ที่มีความรวดเร็วและปลอดภัย

ทักษะด้านการทดสอบอัตโนมัติ (Test Automation) ได้เรียนรู้วิธีการใช้ Playwright ในการเขียนสคริปต์แบบ End-to-End Testing รวมถึงแนวคิด Data-Driven Testing ผ่านไฟล์ JSON

ทักษะการบูรณาการระบบ (System Integration) ได้เรียนรู้วิธีการเชื่อมต่อระบบ Frontend (React), Backend (Go) และ Scripting (Node.js) ให้ทำงานประสานกันเป็นหนึ่งเดียว

ทักษะด้าน Soft Skills ได้พัฒนาทักษะการแก้ปัญหาเฉพาะหน้า (Problem Solving) ทักษะการสื่อสารเพื่อนำเสนองาน และการคิดเชิงตรรกะ (Logical Thinking)

**5.6 ข้อเสนอแนะสำหรับการปฏิบัติงานสหกิจศึกษาสำหรับนักศึกษารุ่นต่อไป**

การเตรียมความพร้อมด้านทักษะพื้นฐาน นักศึกษาควรทบทวนความรู้พื้นฐานด้านการเขียนโปรแกรม (Programming Logic) โครงสร้างข้อมูล (Data Structures) และการทำงานของเว็บแอปพลิเคชัน (HTTP/API) ให้แม่นยำก่อนเริ่มฝึกงาน

การศึกษาคู่มือภาษาอังกฤษ ในการทำงานจริง เทคโนโลยีใหม่ๆ มักจะมีคู่มือเป็นภาษาอังกฤษ นักศึกษาควรฝึกทักษะการอ่านเอกสารทางเทคนิค (Technical Documentation) เพื่อให้สามารถเรียนรู้เครื่องมือใหม่ๆ ได้รวดเร็วขึ้น

ความกล้าที่จะซักถาม เมื่อพบปัญหาที่ใช้เวลาแก้ไขด้วยตนเองนานเกินไป ควรกล้าที่จะสอบถามพี่เลี้ยงหรือเพื่อนร่วมงาน เพราะประสบการณ์จากผู้เชี่ยวชาญจะช่วยชี้แนะแนวทางที่ถูกต้องและประหยัดเวลาได้มาก

ความเปิดกว้างในการเรียนรู้ เทคโนโลยีในโลกธุรกิจเปลี่ยนแปลงอย่างรวดเร็ว นักศึกษาควรมีทัศนคติที่เปิดรับและพร้อมที่จะเรียนรู้ภาษาหรือเครื่องมือใหม่ๆ นอกเหนือจากที่เรียนในห้องเรียนอยู่เสมอ

# บรรณานุกรม


Facebook Open Source. (2567). React: The library for web and native user interfaces. เข้าถึงได้จาก https://react.dev/

Fiber Framework. (2567). Fiber: An Express-inspired web framework written in Go. เข้าถึงได้จาก https://gofiber.io/

Kevin Van Cott. (2567). Mantine React Table: A fully featured Mantine table component built with TanStack Table V8. เข้าถึงได้จาก https://www.mantine-react-table.com/

Microsoft. (2567). Playwright: Fast and reliable end-to-end testing for modern web apps. เข้าถึงได้จาก https://playwright.dev/

OpenJS Foundation. (2567). Node.js. เข้าถึงได้จาก https://nodejs.org/

The Go Authors. (2567). The Go Programming Language. เข้าถึงได้จาก https://go.dev/

# ภาคผนวก

## ภาคผนวก ก. คู่มือการใช้งานระบบ (User Manual)

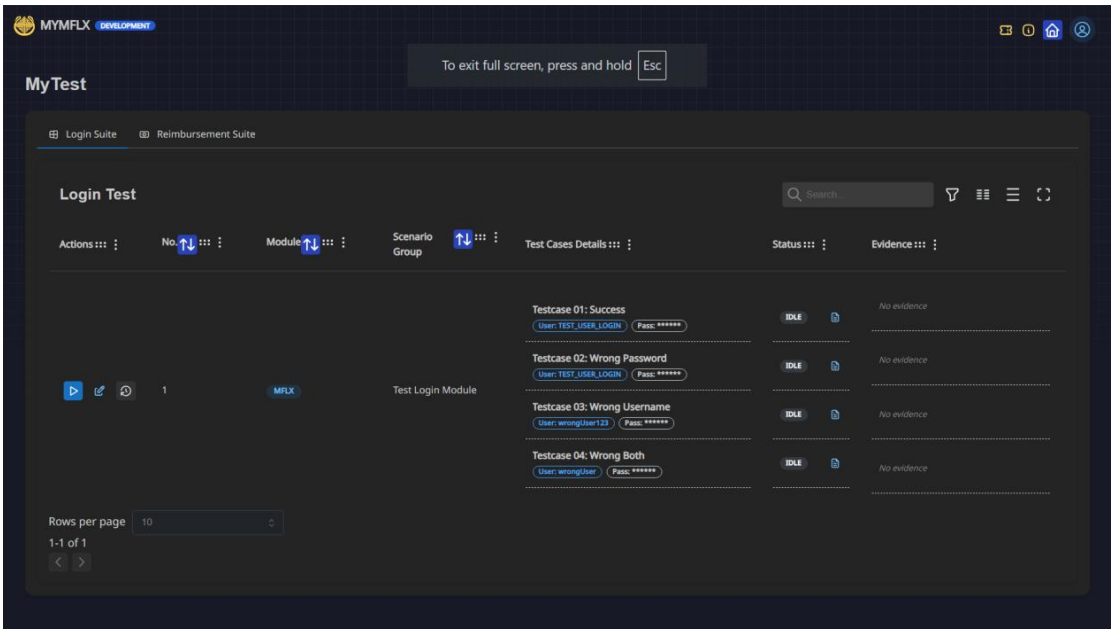
คู่มือฉบับนี้จัดทำขึ้นเพื่ออธิบายขั้นตอนการใช้งาน "ระบบเว็บแอปพลิเคชันสำหรับบริหารจัดการและดำเนินการทดสอบซอฟต์แวร์อัตโนมัติ" สำหรับบุคลากร ผู้ทดสอบระบบ (QA) หรือนักพัฒนาซอฟต์แวร์ โดยมีขั้นตอนการใช้งานหลักดังต่อไปนี้

ขั้นตอนที่ 1 การเข้าสู่ระบบและหน้ากระดานข้อมูลรวม (Dashboard)

1. เปิดเว็บเบราว์เซอร์และเข้าสู่ URL ของระบบจัดการการทดสอบ

2. เมื่อเข้าสู่ระบบ จะพบกับหน้า Dashboard (Test Suites View) ซึ่งแสดงรายการชุดการทดสอบทั้งหมดที่ถูกตั้งค่าไว้ (เช่น Login Suite และ Reimbursement Suite)

3. ผู้ใช้งานสามารถดูรายละเอียดของแต่ละ Test Case สถานะปัจจุบัน (Status) และตรวจสอบจำนวนการทดสอบได้จากตารางข้อมูล (Data Grid)



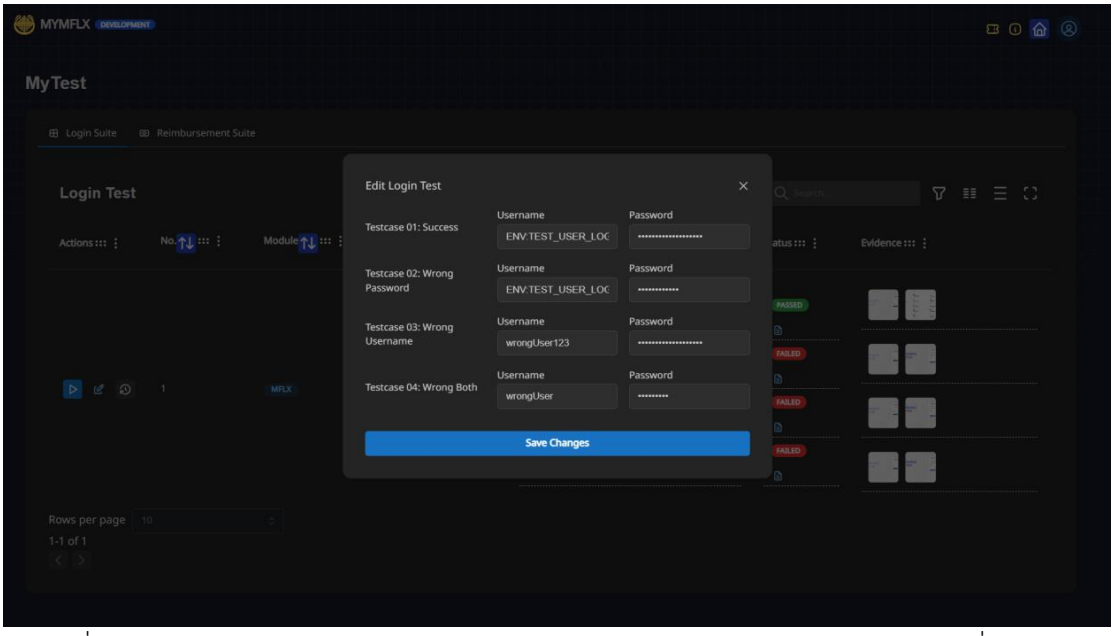

รูปที่ ก.1 หน้ากระดานข้อมูลรวม (Dashboard) แสดงรายการชุดการทดสอบทั้งหมด

ขั้นตอนที่ 2 การแก้ไขข้อมูลชุดการทดสอบ (Edit Test Parameters)

1. ในกรณีที่ต้องการเปลี่ยนตัวแปรที่ใช้ในการทดสอบ เช่น ชื่อผู้ใช้ (Username) หรือรหัสผ่าน (Password) ให้คลิกที่ไอคอน "แก้ไข (Edit)" (สัญลักษณ์รูปดินสอ) ที่อยู่ด้านหน้าชุดการทดสอบ

2. ระบบจะแสดงหน้าต่าง Pop-up ให้ผู้ใช้กรอกข้อมูลใหม่ หรืออ้างอิงจากตัวแปรความลับ (ENV)

3. กดปุ่ม "Save Changes" เพื่อบันทึกการตั้งค่าก่อนเริ่มรันการทดสอบ




รูปที่ ก.2 หน้าต่างสำหรับแก้ไขข้อมูลตัวแปรความลับ (Edit Test Parameters) ก่อนสั่งรันการทดสอบ

ขั้นตอนที่ 3 การสั่งดำเนินการทดสอบ (Run Test)

1. เมื่อตรวจสอบข้อมูลเรียบร้อยแล้ว ให้คลิกที่ปุ่ม "Run" (สัญลักษณ์รูป Play สี่เหลี่ยมสีฟ้า/ส้ม) ในแถวของระบบที่ต้องการทดสอบ

2. ระบบจะส่งคำสั่งไปยังหลังบ้าน (Backend) เพื่อเรียกใช้งานบอทจำลองการทำงาน (Playwright)

3. ระหว่างนี้ สถานะในตารางจะเปลี่ยนเป็นกำลังทำงาน (Running) ผู้ใช้งานสามารถรอจนกว่าระบบจะทำงานเสร็จสิ้น

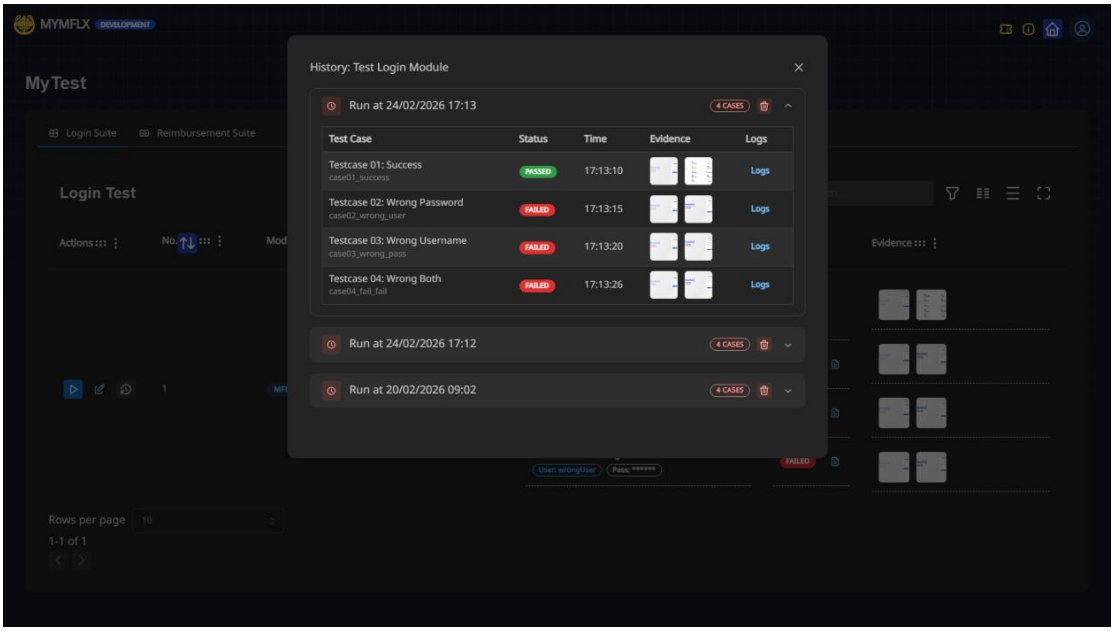
ขั้นตอนที่ 4 การดูประวัติและหลักฐานการทดสอบ (History & Artifacts)

1. เมื่อการทดสอบเสร็จสิ้น ให้คลิกที่ไอคอน "ประวัติ (History)" (สัญลักษณ์รูปนาฬิกาย้อนศร)

2. ระบบจะแสดงหน้าต่างประวัติการรันย้อนหลัง แบ่งตามหมายเลขรอบการทำงาน (Run-ID) และระบุเวลาที่รันอย่างชัดเจน

3. ผู้ใช้งานสามารถดูสถานะสรุปผลว่า PASSED (สีเขียว) หรือ FAILED (สีแดง)

4. หากต้องการตรวจสอบหลักฐาน ให้คลิกที่รูปภาพหน้าจอ (Screenshots) หรือคลิกที่ "Logs" เพื่อดูรายละเอียดการทำงานในขั้นตอนนั้นๆ




**รูปที่ ก.3** หน้าต่างแสดงประวัติการทดสอบย้อนหลัง (History View) พร้อมหลักฐานภาพถ่ายหน้าจอ

## ภาคผนวก ข. ตัวอย่างโค้ดที่สำคัญ (Source Code Snippets)

ในส่วนนี้เป็นการรวบรวมตัวอย่างซอร์สโค้ด (Source Code) บางส่วนที่เป็นหัวใจสำคัญของการทำงานในโครงงาน "ระบบเว็บแอปพลิเคชันสำหรับบริหารจัดการและดำเนินการทดสอบซอฟต์แวร์อัตโนมัติ"

### 1. ซอร์สโค้ดส่วนหลังบ้าน (Backend - Go & Fiber)

ตัวอย่างโค้ดฟังก์ชันสำหรับการรับคำสั่งจากหน้าเว็บ และสั่งรันโปรเซส (Process Spawn) เพื่อเรียกใช้งานบอท Playwright ผ่าน Node.js

```go
import (
	"fmt"
	"os/exec"
	"github.com/gofiber/fiber/v2"
)

// RunTestCase รับ Request จากหน้าบ้านเพื่อสั่งรันเทสเคส
func RunTestCase(c *fiber.Ctx) error {
	caseID := c.Params("caseId")

	// สร้างคำสั่งเรียกใช้งาน Node.js พร้อมส่ง Argument
	cmd := exec.Command("node", "runner.js", "--case", caseID)

	// สั่งรันโปรเซสแบบ Background
	err := cmd.Start()
	if err != nil {
		return c.Status(500).JSON(fiber.Map{
			"status":  "error",
			"message": "Failed to start automation process",
		})
	}

	fmt.Printf("Started Automation Engine for CaseID: [%s]\n", caseID)

	return c.JSON(fiber.Map{
		"status":  "success",
		"message": "Test execution started successfully",
		"run_id":  cmd.Process.Pid,
	})
}
```

**รูปที่ ข.1** ตัวอย่างซอร์สโค้ดส่วนหลังบ้าน (Backend) พัฒนาด้วยภาษา Go สำหรับการรับคำสั่งรันการทดสอบและจัดการโปรเซส

**2. ซอร์สโค้ดส่วนหน้าบ้าน (Frontend - React & Mantine React Table)**

ตัวอย่างโค้ดการเรียกใช้ Mantine React Table เพื่อแสดงผลตารางรายการทดสอบแบบ Data-Driven

```jsx
// components/TestSuiteTable.jsx
import React, { useMemo } from 'react';
import { MantineReactTable } from 'mantine-react-table';
import { Button } from '@mantine/core';
import { IconPlay, IconHistory, IconEdit } from '@tabler/tabler-icons-react';

const TestSuiteTable = ({ data, onRunTest, onOpenHistory, onEdit }) => {
  // กำหนดคอลัมน์ของตาราง
  const columns = useMemo(
    () => [
      { accessorKey: 'testcase_id', header: 'No.' },
      { accessorKey: 'module_name', header: 'Module' },
      { accessorKey: 'scenario_group', header: 'Scenario Group' },
      { accessorKey: 'status', header: 'Status' },
    ],
    []
  );

  return (
    <MantineReactTable
      columns={columns}
      data={data}
      enableRowActions
      positionActionsColumn="first"
      renderRowActions={({ row }) => (
        <div style={{ display: 'flex', gap: '8px' }}>
          <Button color="blue" onClick={() => onRunTest(row.original.testcase_id)
            <IconPlay size={16} />
          </Button>
          <Button color="gray" onClick={() => onEdit(row.original)}>
            <IconEdit size={16} />
          </Button>
          <Button color="teal" onClick={() => onOpenHistory(row.original.testcase
            <IconHistory size={16} />
          </Button>
        </div>
      )}
    />
  );
};

export default TestSuiteTable;
```

รูปที่ ข.2 ตัวอย่างซอร์สโค้ดส่วนหน้าบ้าน (Frontend) พัฒนาด้วย React และไลบรารี Mantine React Table สำหรับแสดงผลตารางรายการทดสอบ

### 3. โครงสร้างไฟล์การตั้งค่าชุดทดสอบ (Automation Engine - JSON Scenario)

ตัวอย่างโครงสร้างไฟล์ JSON (Data-Driven Configuration) ที่บอท Playwright จะอ่านเพื่อนำไปจำลองพฤติกรรมการทดสอบระบบเข้าสู่ระบบ



```
{
  "testCase": {
    "caseId": "case02_wrong_user",
    "steps": [
      {
        "action": "navigate",
        "url": "{{BASE_URL}}/login"
      },
      {
        "action": "fill_login_form",
        "username": "ENV:TEST_USER_LOGIN",
        "password": "ENV:TEST_PASS_LOGIN",
        "description": "1. กรอกรหัสลงในฟอร์ม"
      },
      {
        "action": "screenshot",
        "name": "after_fill_data",
        "description": "2. Screenshot หลังกรอกรหัสเสร็จ"
      },
      {
        "action": "waitForTimeout",
        "ms": 1000
      },
      {
        "action": "click_login_button",
        "description": "3. กดปุ่ม Login"
      }
    ]
  }
}
```

รูปที่ ข.3 ตัวอย่างโครงสร้างไฟล์การตั้งค่าชุดการทดสอบ (JSON Configuration) สำหรับควบคุมการทำงานของบอท Playwright

# ประวัติผู้จัดทำ